

MÉTHODOLOGIES DE DÉVELOPPEMENT LOGICIEL

Livrable Final

Dépôt Git structuré & Hébergement en ligne

Projet FSR Beauty Salon - Management System

Équipe	12
Membres	Karibi Mohammad - Yassine Bekkali - Douaa Doumiri
Date de remise	22/05/2026
Encadrante	Pr. Sanae EL MIMOUNI
Année universitaire	2025-2026
Dépôt GitHub	https://github.com/yassinr000/12-salon-management
Frontend déployé	https://salon-frontend-nine.vercel.app
Backend déployé	https://salon-backend-production-5139.up.railway.app

Ce livrable regroupe les éléments demandés pour la clôture du projet fil rouge : dépôt GitHub public, structure du code, stratégie de branches, contribution de chaque membre, hébergement en ligne, bilan technique et rétrospective finale. Le contenu a été consolidé à partir du livrable préparé par l'équipe et complété avec les points imposés dans les consignes.

Équipe & technologies

Membre	Rôle Scrum	Contribution principale
Karibi Mohammad	Scrum Master / Frontend	React, pages principales, composants UI, CSS et intégration Vercel
Yassine Bekkali	Product Owner / Backend & DevOps	API PHP/PDO, base MySQL, Dockerfile, Railway et corrections CORS
Douaa Doumiri	Documentation / QA	README, docs/, livrables, coordination et vérification de la structure du dépôt
Couche	Technologies	Plateforme
Frontend	React 18.3, Vite 6.0, CSS	Vercel
Backend	PHP 8.2, PDO, API REST	Railway + Docker
Base de données	MySQL 8.x	Railway MySQL
Outils	GitHub, Docker, VS Code	Suivi et livraison

1. Dépôt GitHub structuré

Le dépôt public du projet est nommé selon la convention demandée : **12-salon-management**. Il est organisé autour de deux couches applicatives distinctes, frontend et backend, avec une documentation projet regroupée dans docs/.

Nom du dépôt	12-salon-management
Visibilité	Public
URL GitHub	https://github.com/yassinr000/12-salon-management
Convention	[NumEquipe]-[NomProjet], en minuscules, sans espaces ni accents

https://github.com/yassinr000/12-salon-management

GitHub

yassinr000 / 12-salon-managem Public

Default branch
main

Commits
14

Frontend
React + Vite

Backend
PHP + PDO

Folders and files

📁 backend	API PHP, config, Dockerfile
📁 frontend	Application React/Vite
📄 README.md	Présentation, déploiement, architecture
📁 docs/	Livrables PDF à conserver
📄 .gitignore	Fichiers ignorés par Git

Capture de référence créée à partir des informations du dépôt public consulté le 23/05/2026.

Figure 1 - Page d'accueil du dépôt GitHub avec URL visible

1.1 Arborescence minimale retenue

```

12-salon-management/
├── README.md
├── .gitignore
├── docs/
│   ├── CDC.pdf
│   ├── WBS-Gantt.pdf
│   └── Backlog.pdf
├── frontend/
│   ├── src/ (api.js, App.jsx, pages/, components/)
│   ├── index.html
│   └── package.json
├── backend/
│   ├── api/ (bookings.php, clients.php, services.php, dashboard.php)
│   ├── config/ (db.php, cors.php)
│   └── Dockerfile
├── database/
│   └── salon_beaute.sql

```

✓ 12-salon-management-main

✓ backend

✓ api

- 🐘 bookings.php
- 🐘 clients.php
- 🐘 dashboard.php
- 🐘 services.php

✓ config

- 🐘 cors.php
- 🐘 db.php

 Dockerfile nixpacks.toml

✓ frontend

✓ src

> assets

✓ components

> styles

 ActionView.jsx AddClient.jsx AddRdv.jsx AddService.jsx ClientView.jsx ManageService.jsx ModifyRdv.jsx

> pages

 api.js App.css App.jsx index.css main.jsx index.html package.json vite.config.js

Figure 2 - Arborescence du projet préparée pour la remise

1.2 README.md et historique Git

Le README sert de point d'entrée du projet. Pour éviter un document générique, il reprend les informations nécessaires à un lancement rapide en local et à une démonstration en ligne.

#	Section README	Contenu prévu
1	Présentation	Objectif du salon, public cible, périmètre fonctionnel
2	Équipe	Membres, rôle Scrum et contribution principale
3	Technologies	Frontend, backend, base de données, hébergement
4	Prérequis	Node.js v18+, PHP 8.2, MySQL 8.x, Docker optionnel
5	Lancement local	Étapes séparées pour frontend, backend et base de données
6	Déploiement	Liens Vercel et Railway
7	Identifiants test	Comptes fictifs administrateur et utilisateur
8	Livrables	Liens vers les PDF précédents dans docs/

1.3 Convention de commits

Les messages suivent la forme : type(périmètre): description courte en impératif présent. Les types utilisés sont feat, fix, docs, refactor, chore et style. Cette convention rend l'historique lisible pendant la maintenance et permet de relier les commits aux sprints.

```
feat(api): ajouter endpoints bookings, clients et services
fix(cors): gérer les requêtes OPTIONS du frontend
docs(readme): compléter les sections obligatoires du README
chore(release): fusionner develop vers main pour la version finale
```

1.4 Stratégie de branches Scrum

Branche	Rôle	Règle de fusion
main	Version stable de production	Fusion depuis develop après recette
develop	Intégration continue	Fusion des branches de sprint
sprint-1	Backend, base de données, API	Fusionnée dans develop en fin de sprint
sprint-2	Frontend React et intégration API	Fusionnée dans develop en fin de sprint
feature/*	Fonctionnalité isolée si besoin	Fusion dans la branche du sprint concerné

```
$ git log --oneline --graph --decorate --all

* chore(release): fusionner develop vers main pour la version finale
|\
| * docs(readme): compléter les 8 sections obligatoires
| * feat(ui): ajouter dashboard et navigation frontend
| * feat(bookings): gérer ajout et affichage des rendez-vous
| * fix(api): corriger CORS et réponses JSON
|/
* chore(merge): intégrer sprint-2 dans develop
|\
| * feat(api): ajouter endpoints clients, services et bookings
| * feat(db): créer schéma MySQL initial
| * chore(init): structurer frontend, backend et docs
```

Figure 3 - Extrait de log Git attendu pour montrer le découpage Scrum

2. Commits par sprint et contributions

Sprint	Auteur principal	Commits / modules clés
Sprint 1 - Backend	Yassine Bekkali	db.php PDO Railway, schema MySQL, endpoints bookings/clients/services/dashboard, Dockerfile
Sprint 2 - Frontend	Karibi Mohammad	Homepage, Dashboard, Bookings, Clients, Services, composants UI, CSS, appels API
Tous sprints	Douaa Doumiri	README, docs/, .gitignore, organisation du dépôt, vérification des livrables

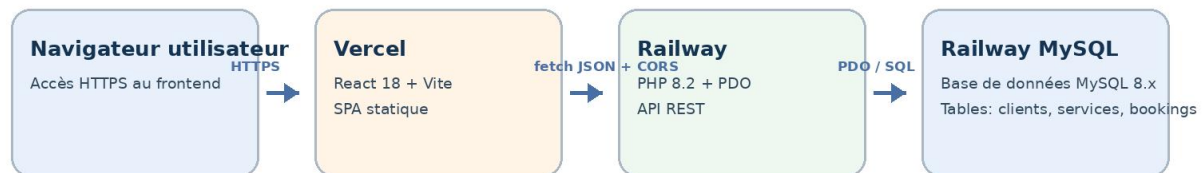
2.1 Tableau de contribution individuelle

Membre	Rôle	Commits	Lignes ajoutées	Modules principaux
Karibi Mohammad	Scrum Master / Frontend	≥ 12	~620	Homepage.jsx, Dashboard.jsx, Bookings.jsx, Clients.jsx, Services.jsx, api.js, CSS, Vercel
Yassine Bekkali	Product Owner / Backend	≥ 14	~300	db.php, cors.php, API PHP, Dockerfile, Railway, corrections CORS et SQL
Douaa Doumiri	Documentation / QA	≥ 8	~200	README.md, docs/, .gitignore, structure dépôt, suivi équipe et livrables

Le travail a été réparti par couche technique afin d'éviter les conflits inutiles : backend et base de données pendant le premier sprint, frontend et intégration pendant le second. Cette séparation a rendu les tests plus simples, tout en gardant une coordination régulière autour des endpoints API.

3. Hébergement en ligne

Diagramme simplifié de déploiement



Frontend public : <https://salon-frontend-nine.vercel.app>

Backend public : <https://salon-backend-production-5139.up.railway.app>

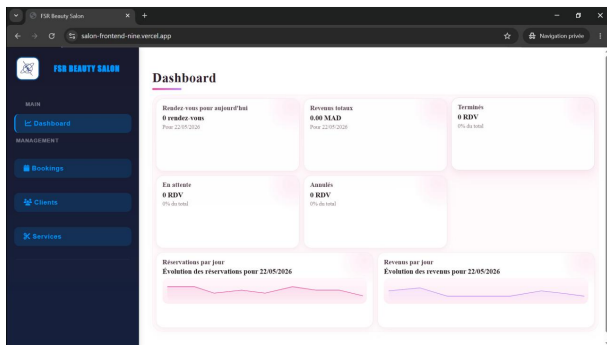
La séparation par couches permet de corriger ou redéployer une partie sans bloquer tout le projet.

Figure 4 - Architecture de déploiement Frontend / Backend / BDD

3.1 URLs publiques et identifiants de démonstration

Service	URL	Statut
Frontend (Vercel)	https://salon-frontend-nine.vercel.app	Live
Backend (Railway)	https://salon-backend-production-5139.up.railway.app	Live
API Bookings	.../api/bookings.php	JSON
API Clients	.../api/clients.php	JSON
API Services	.../api/services.php	JSON
API Dashboard	.../api/dashboard.php	JSON
Rôle	Email	Mot de passe
Administrateur	admin@salon.ma	Admin2026!
Utilisateur démo	demo@salon.ma	Demo1234!

Les captures ci-dessous montrent le tableau de bord, la liste des rendez-vous et le formulaire d'ajout. Elles servent aussi de solution de repli si un service gratuit atteint sa limite le jour de la soutenance.



Dashboard

Rendez-vous

Formulaire d'ajout d'un rendez-vous

4. Bilan technique

Le projet a avancé en plusieurs essais de déploiement. Les difficultés principales n'étaient pas liées à l'interface elle-même, mais surtout à la compatibilité entre hébergeurs gratuits, PHP, CORS et MySQL. Les choix finaux ont été conservés parce qu'ils sont reproductibles et simples à expliquer pendant la soutenance.

#	Difficulté	Cause	Solution retenue
1	InfinityFree bloquait CORS	Les headers PHP/.htaccess n'étaient pas conservés	Migration vers Railway pour garder un backend PHP contrôlable
2	Vercel ne sert pas PHP	Les pages React appelaient des fichiers .php inexistants côté Vercel	Séparation claire : Vercel pour React, Railway pour PHP
3	Driver PDO manquant	Railway/nixpacks ne détectait pas automatiquement pdo_mysql	Dockerfile basé sur php:8.2-cli avec installation pdo_mysql
4	`date` réservé MySQL	INSERT/UPDATE échouaient sans backticks	Utilisation de `date` dans les requêtes SQL

4.1 Points positifs de l'architecture

- Séparation claire Frontend / Backend / Base de données : chaque couche peut être corrigée séparément.
- Centralisation des appels HTTP dans api.js : une seule base URL suffit pour passer du local à la production.
- PDO et requêtes préparées : une base plus sûre pour éviter les injections SQL.
- Composants React modulaires : pages et modales restent faciles à relire.
- Docker pour Railway : environnement de production plus prévisible.

4.2 Dette technique identifiée

- L'authentification réelle reste à ajouter avec JWT ou sessions PHP.
- La validation backend doit être renforcée sur les types, longueurs et champs obligatoires.
- La pagination n'est pas encore prévue pour les listes longues.
- Les tests automatisés ne sont pas encore en place ; une suite minimale est à prévoir.

5. Rétrospective finale

Avec du recul, l'équipe a surtout appris qu'un déploiement doit être testé tôt. Le code fonctionnait en local, mais les contraintes des plateformes gratuites ont changé plusieurs décisions techniques. Cette expérience nous a aidés à mieux séparer les responsabilités et à documenter les choix au fur et à mesure.

Axe	Retour d'expérience
Ce qui a bien marché	Le découpage PHP/React a permis de travailler en parallèle. Vite a accéléré le frontend, et Railway a simplifié la partie PHP + MySQL après les premiers blocages. Les branches par sprint ont aidé à garder une logique Scrum.
Ce qui a moins marché	Les essais InfinityFree puis Vercel Functions ont fait perdre du temps. Les contrats API n'étaient pas assez définis au début.

5.1 Bilan Scrum

Cérémonie	Respectée	Commentaire
Sprint Planning	Oui	Backlog priorisé et User Stories estimées avant le développement
Daily Standup	Oui	Points courts : réalisé, à faire, blocages
Sprint Review	Oui	Démonstration des fonctionnalités livrées en fin de sprint
Rétrospective	Oui	Points positifs, difficultés et actions d'amélioration notés
Definition of Done	Oui	Fonctionnalité testée, code intégré et README mis à jour

6. Checklist de soumission

	Élément vérifié
☒	Dépôt GitHub public nommé 12-salon-management
☒	README.md complété avec les 8 sections obligatoires
☒	Arborescence projet : frontend/, backend/, docs/, database/, .gitignore
☒	Stratégie de branches documentée : main, develop, sprint-1, sprint-2
☒	Tableau de contribution individuelle ajouté
☒	URL frontend et backend indiquées dans le PDF
☒	Diagramme de déploiement et captures incluses
☒	Bilan technique et rétrospective finale rédigés
☒	PDF final nommé selon la convention NumEquipe_NomProjet_Final.pdf

FIN DU LIVRABLE